

Tutor

Mauro Farina

- Dottorando in ingegneria industriale e dell'informazione
- 2° anno
- Misure di Internet
- mauro.farina@phd.units.it

Repository esercitazioni

- github.com/mauro-farina/database-2026

Esercizio 12

Per ogni studente che ha migliorato la propria media (ponderata) almeno una volta, restituire il numero di volte in cui ciò è avvenuto

- **Non** è richiesto che l'esercizio sia svolto con un'unica query

Risultato atteso (248 righe), in ordine alfabetico di matricola sono:

- EC2100025 6
- EC2100142 5
- EC2100149 8
- ...
- SM3500963 1
- SM3500986 3

Esercizio 12

Per ogni studente che ha migliorato la propria media (ponderata) almeno una volta, restituire il numero di volte in cui ciò è avvenuto

- **Suggerimento:** creare la vista `media_nel_tempo(studente, data, media)` che rappresenti la media ponderata di ciascuno studente man mano che sostiene gli esami

Risultato atteso (248 righe), in ordine alfabetico di matricola sono:

- EC2100025 6
- EC2100142 5
- ...
- SM3500963 1
- SM3500986 3

Esercizio 12: Soluzione (1)

Restituire il numero di volte che ciascuno studente ha migliorato la propria media dopo aver sostenuto un nuovo esame: (1) `media_nel_tempo`

```
CREATE VIEW media_nel_tempo AS
SELECT e.studente, e.data,
       (SELECT sum(e2.voto * c2.cfu) / sum(c2.cfu)
        FROM esami e2
        INNER JOIN corsi c2 ON c2.codice = e2.corso
        WHERE e2.studente = e.studente
              AND e2.data <= e.data) AS media
FROM esami e GROUP BY e.studente, e.data;
```

Esercizio 12: Soluzione (2)

Restituire il numero di volte che ciascuno studente ha migliorato la propria media dopo aver sostenuto un nuovo esame: (2) Risultato

```
SELECT m1.studente, count(*) FROM media_nel_tempo m1
WHERE m1.media > (
    SELECT m2.media FROM media_nel_tempo m2
    WHERE m2.studente = m1.studente
    AND m2.data < m1.data
    ORDER BY m2.data DESC LIMIT 1
)
GROUP BY m1.studente;
```

Esercizio 13

Elencare nome e cognome di tutti gli studenti che hanno una media (aritmetica) superiore alla media (aritmetica) del proprio corso di laurea

Risultato atteso (149 righe):

- Adamo Baldi
- Adamo Bianchi
- Adamo Bonventre
- ...
- Vida Vinciguerra
- Coe Rudel

Esercizio 13: Soluzione

Elencare nome e cognome di tutti gli studenti che hanno una media (aritmetica) superiore alla media (aritmetica) del proprio corso di laurea

```
SELECT DISTINCT s.nome, s.cognome FROM studenti s
JOIN esami e1 ON e1.studente = s.matricola
JOIN corsi c ON c.codice = e1.corso
GROUP BY s.matricola, s.nome, s.cognome
HAVING avg(e1.voto) > (
    SELECT avg(e2.voto) FROM esami e2
    WHERE substring(e2.studente, 1, 4) =
        substring(s1.matricola, 1, 4)
);
```

Esercizio 14

Si dice che uno studente *S* ha "*brillato*" con un professore *P* se *S* ha

- sostenuto l'esame di tutti i corsi tenuti da *P*, e
- in ciascuno di essi ha ottenuto un voto maggiore alla media del corso.

Esercizio:

1. Scrivere la funzione `brillato(S, P)` che restituisce `TRUE` se *S* ha "*brillato*" con *P*, altrimenti `FALSE`.
2. Restituire il numero di studenti che hanno "*brillato*" con più di 5 professori

Risultato atteso: 25


```
CREATE FUNCTION brillato(  
    stud CHAR(9),  
    prof INT  
)  
  
RETURNS BOOL READS SQL DATA  
BEGIN  
    DECLARE n_corsi INT;  
    DECLARE n_brillati INT;  
  
    SELECT count(*)  
    INTO n_corsi  
    FROM corsi  
    WHERE professore = prof;  
  
    SELECT count(*)  
    INTO n_brillati ...
```

```
...  
FROM esami e  
INNER JOIN corsi c  
ON e.corso = c.codice  
WHERE e.studente = stud  
    AND c.professore = prof  
    AND e.voto > (  
        SELECT AVG(e2.voto)  
        FROM esami e2  
        WHERE e2.corso = e.corso  
    );  
  
RETURN n_corsi > 0  
    AND n_corsi = n_brillati;  
  
END
```

Esercizio 14: Soluzione (2)

Restituire il numero di studenti che hanno "*brillato*" con più di 5 professori

```
SELECT count(*) FROM (  
    SELECT 1  
    FROM studenti s  
    INNER JOIN esami e ON s.matricola = e.studente  
    INNER JOIN corsi c ON c.codice = e.corso  
    WHERE brillato(s.matricola, c.professore) IS TRUE  
    GROUP BY s.matricola  
    HAVING count(DISTINCT c.professore) > 5  
) AS tab;
```

Esercizio 14 bis

Si dice che uno studente S ha "*brillato*" con un professore P se S ha

- sostenuto l'esame di tutti i corsi tenuti da P , e
- in ciascuno di essi ha ottenuto un voto maggiore alla media del corso.

Restituire, **con un'unica query**, il numero di studenti che hanno "*brillato*" con più di 5 professori.

Esercizi extra

1. Tutti i corsi per i quali nessuno ha passato l'esame a gennaio 2020
2. Se 1 CFU sono 8 ore di lezione, qual è il professore (tra quelli che insegnano almeno un corso) che passa meno tempo a insegnare
3. Creare un prepared statement che restituisca l'elenco di studenti iscritti a un corso di laurea (dato in input)
4. Scrivere la UDF `rank_cdl` che restituisca il rank (output) di uno studente (input) nel suo corso di laurea (cdl) in base alla sua media ponderata:
 - a. Rank 1 → studente con media ponderata più alta
5. Supponendo si possano rifare gli esami, scrivere un trigger che verifichi che il nuovo voto non sia più basso di quello già registrato. In tal caso, lanciare un errore

Esercizio 15

Creare la stored procedure `sp_trend_esame(codice_corso CHAR(5))` che elenchi nome e cognome di tutti gli studenti che hanno passato l'esame, aggiungendo la colonna `Trend` che indica se lo studente ha preso un voto "Sopra", "Sotto" o "Uguale" alla media voti del corso.

Risultato atteso per `CALL sp_trend_esame("079IN");`

- Enea Ghini Sopra
- Diego Vaccaro Sopra
- Sofia Cagnotti Sopra
- Federico Pastore Sotto

```
CREATE PROCEDURE
    sp_trend_esame(
        IN COD_CORSO CHAR(5)
    )
BEGIN
    DECLARE avg_voto DOUBLE;

    SELECT avg(voto)
    INTO avg_voto FROM esami
    WHERE corso = COD_CORSO;

    (☆) = "
        studenti
        JOIN esami
        ON studente = matricola
        WHERE corso = COD_CORSO
    "
```

```
SELECT nome, cognome,
        'Sopra' AS Trend
FROM ☆
        AND voto > avg_voto

UNION

SELECT nome, cognome,
        'Sotto' AS Trend
FROM ☆
        AND voto < avg_voto

UNION

SELECT nome, cognome,
        'Uguale' AS Trend
FROM ☆
        AND voto = avg_voto;

END
```